Escola Superior de Tecnologia — Instituto Politécnico do Cávado e do Ave
Licenciatura em Engenharia de Sistemas Informáticos

Sentinel-AI (EDCA)

ETL Data Correlation Agent — integração, correlação e análise de eventos de segurança física com Pentaho Data Integration, regras e IA local

Unidade Curricular	**Integração de Sistemas de Informação**
Trabalho	Trabalho Prático I — processos ETL
Autor	**Rodrigo Lopes Ferreira**, n.º **31525**
Docentes	Luís Ferreira, Óscar Ribeiro
Ano letivo	2026/27
Data	22 de outubro de 2026

Repositório: https://github.com/LFtech6/tp01-31525-EDCA

Índice

1. Enquadramento
2. Problema
3. Estratégia utilizada
   3.1 Arquitetura e responsabilidades
   3.2 As três fases ETL
   3.3 Operadores utilizados
   3.4 Dados de teste e avaliação
   3.5 Segurança e configuração
   3.6 Organização do projeto e da entrega
4. Transformações
   4.1 tr_load_reference
   4.2 tr_extract_events
   4.3 tr_correlate_rules
   4.4 tr_ai_analysis (+ serviço /ai/analyze)
   4.5 tr_evaluate
   4.6 tr_export_incidents
   4.7 tr_build_notification
5. Jobs
   5.1 jb_sentinel_main
   5.2 Registo de execuções e logs
   5.3 Notificação por e-mail
   5.4 Orquestração a partir do dashboard (API /ops + Node-RED)
6. Vídeo com demonstração (QR Code)
7. Conclusão e trabalhos futuros
8. Referências

1. Enquadramento

Equipa	Trabalho individual: Rodrigo Lopes Ferreira (n.º 31525)
Unidade Curricular	Integração de Sistemas de Informação (ISI), 2026/27
Curso	Licenciatura em Engenharia de Sistemas Informáticos (LESI), EST — IPCA
Tema	Correlação de eventos de segurança física (controlo de acessos + sensores) com ETL, motor de regras e IA local
Plataforma ETL principal	Pentaho Data Integration (Kettle) 10.2.0.0-222 — Spoon, Pan e Kitchen
Ferramentas complementares	PostgreSQL 14; Python 3.12 + FastAPI; Ollama com o modelo `qwen3:4b-instruct`; Node-RED 5 + FlowFuse Dashboard 2.0; servidor SMTP (Office 365)
Ambiente	macOS (Apple M2, 8 GB RAM), todos os serviços locais (sem Docker)

O trabalho enquadra-se no cenário de **auditoria de segurança** e de **reação a eventos** referido na motivação do enunciado. Numa organização, o controlo de acessos (leitores de cartão) e os sensores (movimento, portas) são normalmente sistemas distintos, de fabricantes diferentes, com formatos e qualidade de dados diferentes. Cada sistema, isolado, vê apenas uma parte do que aconteceu. O valor está em **integrar e cruzar** esses dados com a informação de referência (zonas, horários, utilizadores, permissões).

O processo foi desenvolvido para demonstrar as três fases obrigatórias (E, T e L) e os critérios de mais-valia do enunciado. Entre eles estão a diversidade de formatos (CSV, JSON, XML e YAML), as expressões regulares, os joins e lookups, a orquestração por Jobs, os logs, o acesso a serviços remotos (REST e e-mail), as operações sobre bases de dados e a visualização num dashboard.

2. Problema

Pretende-se demonstrar que um processo ETL consegue transformar registos dispersos e "sujos" de dois sistemas de segurança em **casos de segurança** com contexto, e que esses casos podem ser classificados de duas formas independentes e comparáveis:

1. **Abordagem A — motor de regras determinístico**, implementado no próprio Pentaho (regras explícitas com pesos);
2. **Abordagem B — análise contextual por um modelo de linguagem local** (Qwen3 4B via Ollama), chamado pelo Pentaho através de um serviço REST, que devolve uma classificação estruturada com explicação, ação recomendada e as evidências usadas.

Exemplos do tipo de situação que se quer detetar:

- um acesso recusado seguido de movimento dentro da zona (alguém entrou por outra via);
- várias recusas seguidas numa sala de servidores;
- um acesso concedido a quem não tem permissão;
- uma credencial de um ex-colaborador ainda em uso;
- *tailgating* (entrar "à boleia" de outra pessoa);
- movimento noturno sem qualquer acesso registado.

O problema também tem a dificuldade inversa: evitar falsos alarmes em situações legítimas. Por exemplo, um operador do turno da noite, uma ronda de segurança ou um cartão mal lido à primeira.

Os dados de origem têm problemas de qualidade propositados, que o ETL tem de resolver:

- identificadores em formatos diferentes (`u-41`, ` U012`, `user15`);
- datas em três formatos e com fuso horário;
- estados escritos de várias formas (`denied`, `Denied`, `DENIED`);
- booleanos heterogéneos (`true`, `yes`, `1`, `on`);
- registos duplicados, campos obrigatórios vazios e datas impossíveis (`2026-13-45`).

Os registos inválidos não podem simplesmente desaparecer: têm de ficar registados com o motivo da rejeição.

Objetivos mensuráveis.

1. Integrar as 4 fontes num modelo único e rastreável.
2. Correlacionar os eventos em casos, sem misturar cenários diferentes.
3. Classificar os casos por regras e por IA.
4. Medir as duas abordagens contra um *ground truth* definido **antes** das regras (precisão, recall, F1).
5. Exportar os incidentes, notificar por e-mail e disponibilizar tudo num dashboard que permita executar o processo.

3. Estratégia utilizada

3.1 Arquitetura e responsabilidades

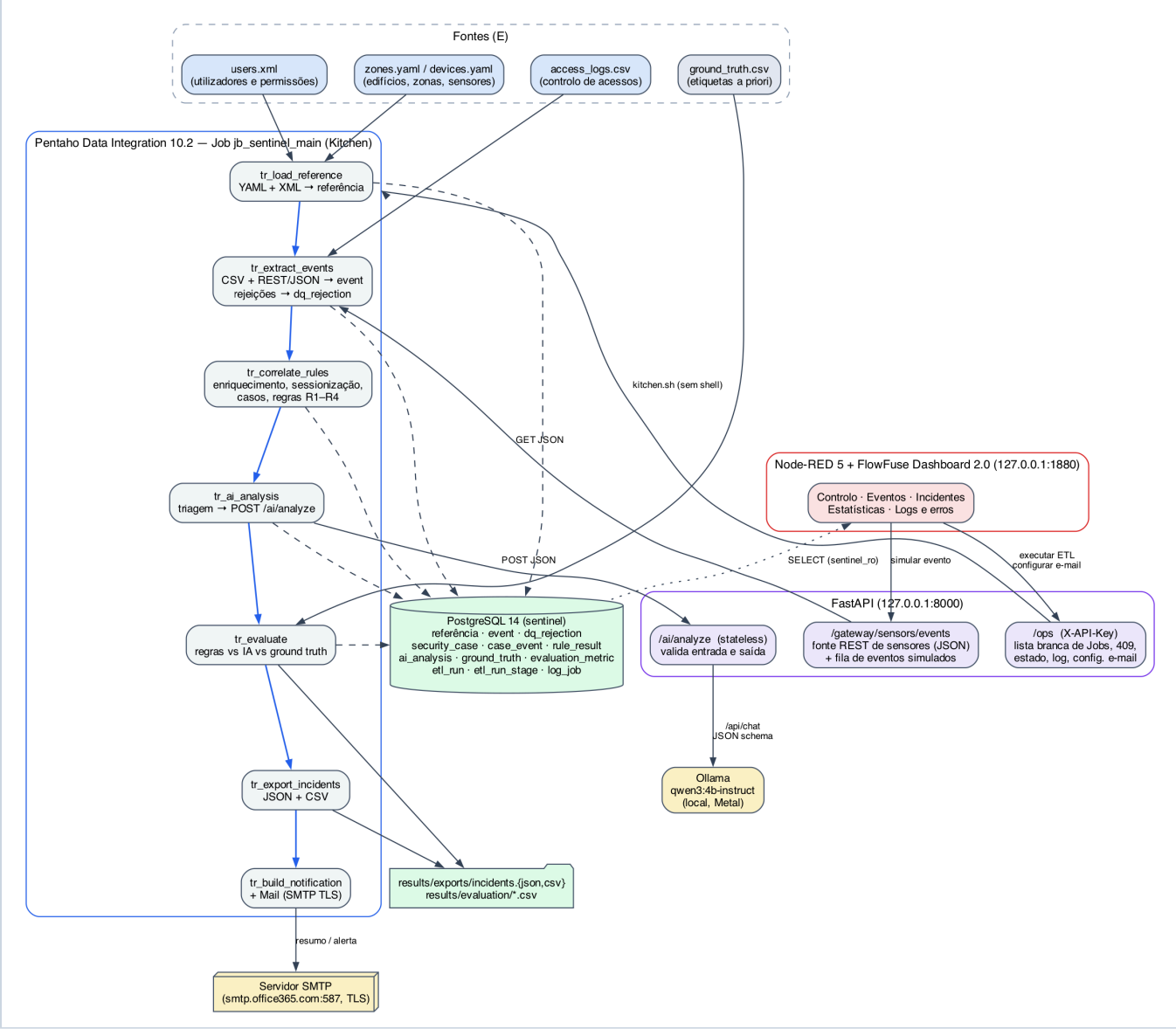


Figura 1 — Arquitetura do Sentinel-AI. O Pentaho é o motor ETL central; os restantes componentes têm papéis delimitados.

Componente	Responsabilidade	Não faz
Pentaho (Kettle)	Toda a extração, transformação, correlação, regras, chamada à IA, avaliação, carregamento, exportação e notificação; Job de orquestração	Interface com o utilizador
FastAPI `/gateway`	Simula o *gateway* IoT dos sensores (fonte REST/JSON) e recebe eventos simulados pelo dashboard	Transformar dados
FastAPI `/ai`	Inferência: valida a entrada, chama o Ollama, valida a saída e devolve JSON (*stateless*)	Escrever na base de dados
FastAPI `/ops`	Lança o Job (lista branca), controla a concorrência, expõe estado, etapas e log, e guarda a configuração de e-mail	Executar comandos arbitrários
Ollama + Qwen3	Classificação contextual e explicação de um caso já correlacionado	Correlacionar ou integrar dados
PostgreSQL	Persistência dos dados, resultados, estado das execuções e logs (16 tabelas)	—
Node-RED + FlowFuse Dashboard 2.0	Controlo e visualização low-code: executar o ETL, ver etapas, eventos, incidentes, estatísticas e erros	Qualquer transformação ETL

Princípio seguido: **os dados só são escritos pelo Pentaho**. O dashboard lê a base de dados com um utilizador apenas de leitura (`sentinel_ro`) e só atua através da API. A IA não decide relações entre eventos: recebe um caso já construído pelo ETL e devolve uma interpretação.

3.2 As três fases ETL

Fase	O que é feito
E — Recolha	• **YAML:** edifícios, zonas e horários (`zones.yaml`) e dispositivos (`devices.yaml`). • **XML:** utilizadores com permissões aninhadas (`users.xml`). • **CSV:** registos do controlo de acessos (`access_logs.csv`, 99 linhas). • **JSON via API REST:** eventos de sensores (`GET /gateway/sensors/events`, 79 documentos). • **CSV:** *ground truth*. • **Base de dados relacional:** lookups em PostgreSQL.
T — Transformação	• Limpeza e validação: *trim*, maiúsculas e minúsculas, nulos, duplicados, validação por expressões Java. • Expressões regulares para normalizar identificadores e datas. • Mapeamento de valores e conversão de tipos. • Lookups (*Stream lookup* e *Database lookup*) e *Merge join*. • Ordenação, sessionização com estado (JavaScript) e agregação (*Group by*). • Cálculo de *hash* e reestruturação de schemas (XML aninhado → linhas; linhas → payload JSON). • Motor de regras e chamada à IA.
L — Persistência e exportação	• *Insert*/*Update* idempotente e *Table output* em PostgreSQL. • Rejeições de qualidade em `dq_rejection`. • Exportação de incidentes em JSON e CSV, e de métricas e comparação em CSV. • Notificação automática por e-mail (resumo com anexos, ou alerta de erro). • Dashboard em tempo real (Node-RED).

3.3 Operadores utilizados

As 7 transformações somam **148 steps** de **31 tipos diferentes**; o Job tem 28 entradas de 9 tipos. Contagem extraída dos ficheiros `.ktr`:

Categoria	Steps PDI (n.º de ocorrências)
Entrada	CSV file input (2), YAML input (2), Get data from XML (2), JSON input (2), REST client (2), Table input (8), Generate rows (1)
Limpeza / normalização	String operations (6), Replace in string com Regex (5), Value mapper (3), If field value is null (1), Calculator (6), Select values com metadados/conversão (22)
Validação / fluxo	User Defined Java Expression (23), Filter rows (3), Block until steps finish (4), tratamento de erros por step (*error hops*, 3)
Duplicados / ordenação	Unique rows (3), Unique rows (HashSet) (2), Sort rows (7)
Junção / cruzamento	Stream lookup (3), Database lookup (4), Merge join (2)
Agregação / cálculo	Group by (3), Modified JavaScript value (1), Get variables (8), Set variables (1)
Saída	Insert/Update (9), Table output (7), Execute SQL script (2), Text file output (3), JSON output (1)
Job	Start, Transformation (8), SQL (12), Check webservice availability, Add filenames to result, Mail (2), Write to log, Success, Abort

3.4 Dados de teste e avaliação

Os dados são sintéticos e determinísticos (semente fixa), gerados por `data/generator/generate_events.py` para três dias (12 a 14 de outubro de 2026). Contêm tráfego normal e **15 cenários** com etiqueta definida **a priori**: 7 suspeitos (S01–S07) e 8 benignos difíceis (B01–B08). A etiqueta é atribuída **ao evento** (`ground_truth.csv`, 152 eventos), não ao caso. Assim, a correlação não pode "escolher" a sua própria resposta certa. Um caso é suspeito se contiver algum evento de um cenário suspeito.

Para a avaliação ser justa:

- a IA nunca recebe o veredicto das regras, o nome do cenário nem o `case_id`;
- os casos enviados à IA são escolhidos por **factos** (recusas, fora de horas, sem permissão, utilizador inativo, fora do turno) e não pelo resultado das regras;
- juntam-se 5 casos de controlo sem sinais, para medir falsos positivos em tráfego comum;
- o *prompt* (v1) foi fixado antes de observar resultados.

3.5 Segurança e configuração

- **Credenciais:** estão num ficheiro `.env` ignorado pelo Git. O Pentaho recebe-as através de um `KETTLE_HOME` local ao projeto (`kettle.properties` gerado a partir do `.env`). Os `.ktr` e o `.kjb` só contêm variáveis como `${PG_PASSWORD}` e `${SMTP_PASSWORD}`.
- **Exposição na rede:** todos os serviços escutam apenas em `127.0.0.1`.
- **API `/ops`:**
  - exige o cabeçalho `X-API-Key`;
  - aceita apenas um nome lógico de Job (lista branca) e lança o Kitchen sem *shell*, com uma lista de argumentos;
  - controla a concorrência com um índice único parcial (HTTP 409 se já houver uma execução ativa);
  - valida o `run_id` contra *path traversal*.

- **Dashboard:** lê a base de dados com um utilizador apenas de leitura.
- **Configuração de e-mail no dashboard:** é validada, guardada só em memória e chega ao Kitchen por variáveis de ambiente. A password nunca é devolvida.

3.6 Organização do projeto e da entrega

O projeto está organizado por responsabilidade: cada pasta corresponde a uma peça da arquitetura (Figura 1). O repositório Git e a pasta `src/` da entrega têm exatamente o mesmo conteúdo.

Pasta / ficheiro	Conteúdo	Peça
`etl/transformations/`	As 7 transformações (`tr_load_reference`, `tr_extract_events`, `tr_correlate_rules`, `tr_ai_analysis`, `tr_evaluate`, `tr_export_incidents`, `tr_build_notification`)	Pentaho
`etl/jobs/`	`jb_sentinel_main.kjb`, o Job de orquestração	Pentaho
`etl/sql/`	SQL mais extenso usado pelos steps *Table input*: seleção e payload dos casos para a IA; composição do e-mail	Pentaho
`etl/kettle-home/`	`KETTLE_HOME` local, com o `kettle.properties` gerado a partir do `.env`. Não vai para o Git nem para a entrega, porque contém credenciais.	Pentaho
`data/raw/`	Fontes: `access_logs.csv`, `sensor_events.json` (servido pela API), `users.xml`, `zones.yaml`, `devices.yaml`	Dados de entrada
`data/test_scenarios/`	`ground_truth.csv` (etiqueta por evento) e `scenarios.md` (catálogo dos 15 cenários)	Avaliação
`data/generator/`	`generate_events.py`: gerador determinístico (semente fixa) dos dados de teste	Dados de entrada
`data/inbox/`	Fila dos eventos simulados no dashboard. É criada quando se usa a simulação e não vai para o Git.	API (gateway)
`database/`	`00_bootstrap.sql` (utilizadores `sentinel` e `sentinel_ro` e a BD) e `01_schema.sql` (16 tabelas; pode ser executado várias vezes)	PostgreSQL
`ai-engine/app/`	Serviço FastAPI: • `main.py`; • `routers/gateway.py`, `routers/ai.py`, `routers/ops.py`; • `llm.py` (prompt e chamada ao Ollama); • `schemas.py` (validação); • `security.py` (`X-API-Key`); • `config.py` (leitura do `.env`).	API / IA
`ai-engine/tests/`	34 testes automáticos (pytest), com o Ollama simulado	API / IA
`node-red/`	`flows.json` (os 5 fluxos/páginas), `settings.js` (só 127.0.0.1), `package.json` (versões fixas). As dependências (`node_modules/`) instalam-se com `npm`.	Node-RED / Dashboard
`scripts/`	Arranque e operação: • `init_env.sh`, `check_env.sh`; • `db_bootstrap.sh`, `sql.sh`;	Operação

	• `kettle_env.sh`, `pdi.sh` (Spoon/Pan/Kitchen com o `KETTLE_HOME` do projeto); • `run_pipeline.sh`, `run_api.sh`, `run_nodered.sh`; • `package_delivery.sh`.	
`results/exports/`, `results/evaluation/`	Saídas do ETL: incidentes (JSON e CSV), métricas e comparação regras vs IA (CSV)	Dados de saída
`results/logs/`	Log completo do Kitchen de cada execução (`<RUN_ID>.log`; não vai para o Git)	Logs
`docs/`	Arquitetura, documentação de cada transformação e do Job, avaliação, diagramas e este relatório	Documentação
`.env` / `.env.example`	Definições e segredos (fora do Git) / modelo sem segredos (no Git)	Configuração

Entrega (`31525.zip`). Segue a organização sugerida pelos docentes. As pastas `dataintegration/` e `data/` são cópias, para consulta direta, dos ficheiros que estão em `src/`. O projeto executa-se a partir de `src/`, onde os caminhos usados pelos scripts e pelas variáveis do Pentaho (`${SENTINEL_DATA}`, `${SENTINEL_RESULTS}`) se mantêm válidos.

```
tp01-31525/
├── README.md             autor, vídeo, descrição dos ficheiros, como executar
├── doc/31525_doc.pdf     este relatório
├── dataintegration/      transformations/ (7 .ktr) · jobs/ (1 .kjb) · sql/ (2 .sql)
├── data/
│   ├── input/            CSV, JSON, XML, YAML + test_scenarios/ (ground truth)
│   └── output/           exports/ · evaluation/ · database/sentinel_dump.sql
└── src/                  projeto completo e executável (= repositório Git)
```

O ficheiro `sentinel_dump.sql` é a exportação da base de dados (`pg_dump`), com a estrutura e os dados de todas as tabelas após a última execução. Permite consultar ou recriar a BD sem executar o ETL.

O pacote é gerado por `scripts/package_delivery.sh` a partir do último commit (`git archive`), o que garante que o `.env`, o `kettle-home` e as dependências nunca entram. O histórico Git completo segue em `src/31525.bundle` (recupera-se com `git clone 31525.bundle`). No fim, o script procura no pacote e em todo o histórico Git as passwords e tokens do `.env` e falha se encontrar algum.

4. Transformações

Os diagramas desta secção foram gerados a partir do XML de cada `.ktr`, nas mesmas posições do *canvas* do Spoon. Cores por família de step: azul = entrada, verde = saída, amarelo = *lookup/join*, lilás = fluxo, ordenação e agregação, cinza = transformação de valores. Hops de erro a vermelho tracejado. Todas as transformações recebem o parâmetro `RUN_ID` do Job e usam a ligação parametrizada `sentinel_pg`.

4.1 tr_load_reference — dados de referência (YAML + XML)

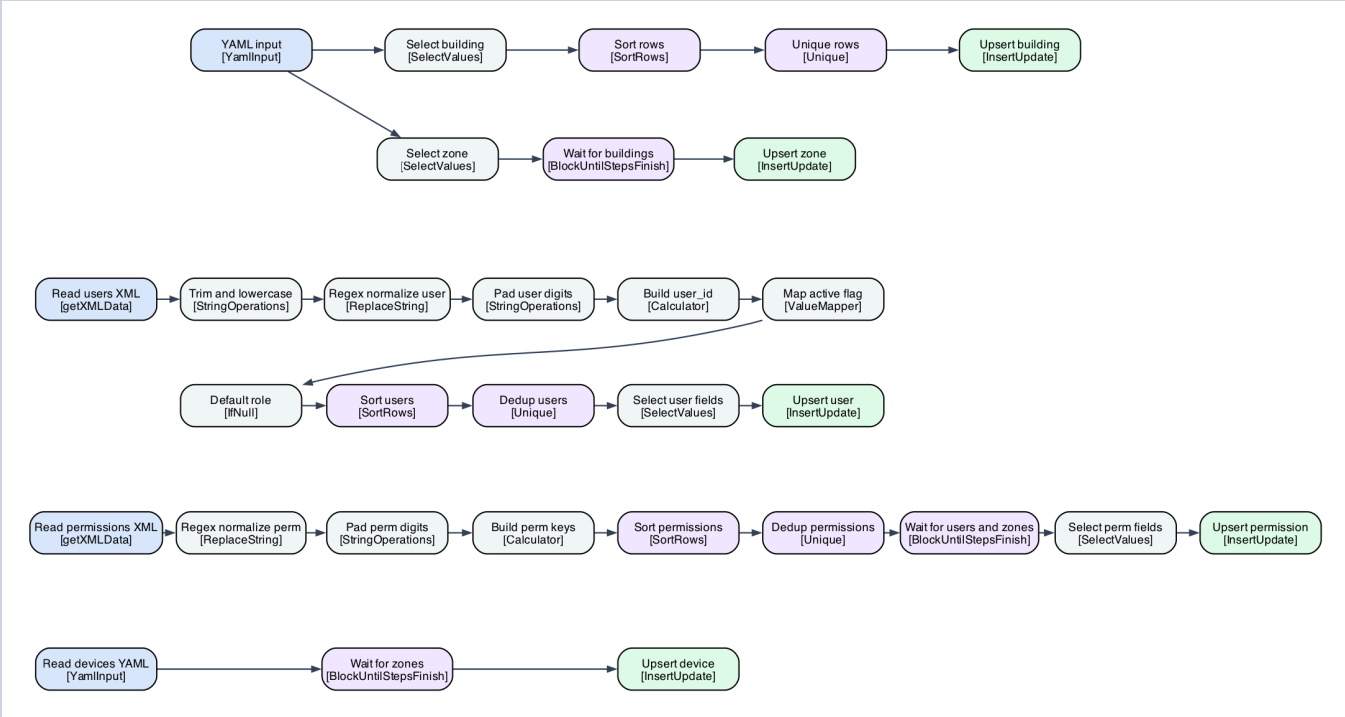


Figura 2 — tr_load_reference (31 steps).

Objetivo. Carregar os dados mestre usados na correlação: 2 edifícios, 8 zonas (com nível de segurança e horário), 24 dispositivos, utilizadores e as respetivas permissões por zona.

Explicação. Há cinco ramos paralelos:

1. **Edifícios:** *YAML input* → *Select values* → *Sort* + *Unique rows* → *Insert/Update*.
2. **Zonas:** os horários passam de texto a `Date` com a máscara `HH:mm`, para a comparação do *Insert/Update* funcionar na reexecução.
3. **Utilizadores** (*Get data from XML*, XPath `/users/user`):
   - *String operations*: *trim* e minúsculas;
   - **Regex** `[^0-9]` → "" mais *padding* e prefixo, para que `u-7`, `U012` e `user15` passem a `U007`, `U012` e `U015`;
   - **Regex** `^(\d):` → `0$1:`, para que `8:00` passe a `08:00`;
   - *Value mapper*: `true` / `yes` / `1` → `Y`;
   - *If field value is null*: papel vazio → `unassigned`;
   - *Unique rows*: remove o utilizador U003 duplicado.
4. **Permissões:** a lista aninhada `allowed_zones/zone` é reestruturada numa linha por par utilizador×zona (XPath com `../../@id`), e `z3` passa a `Z03`.
5. **Dispositivos:** *YAML input* → *Insert/Update*.

Os steps *Block until steps finish* garantem a ordem exigida pelas chaves estrangeiras (zonas depois de edifícios, permissões depois de utilizadores e zonas).

Resultado verificado. 2 edifícios, 8 zonas, 24 dispositivos, 15 utilizadores e 49 permissões. Na segunda execução o *Insert/Update* não insere nem atualiza nada (O=0, U=0), ou seja, a transformação é idempotente.

4.2 tr_extract_events — eventos (CSV + API REST/JSON)

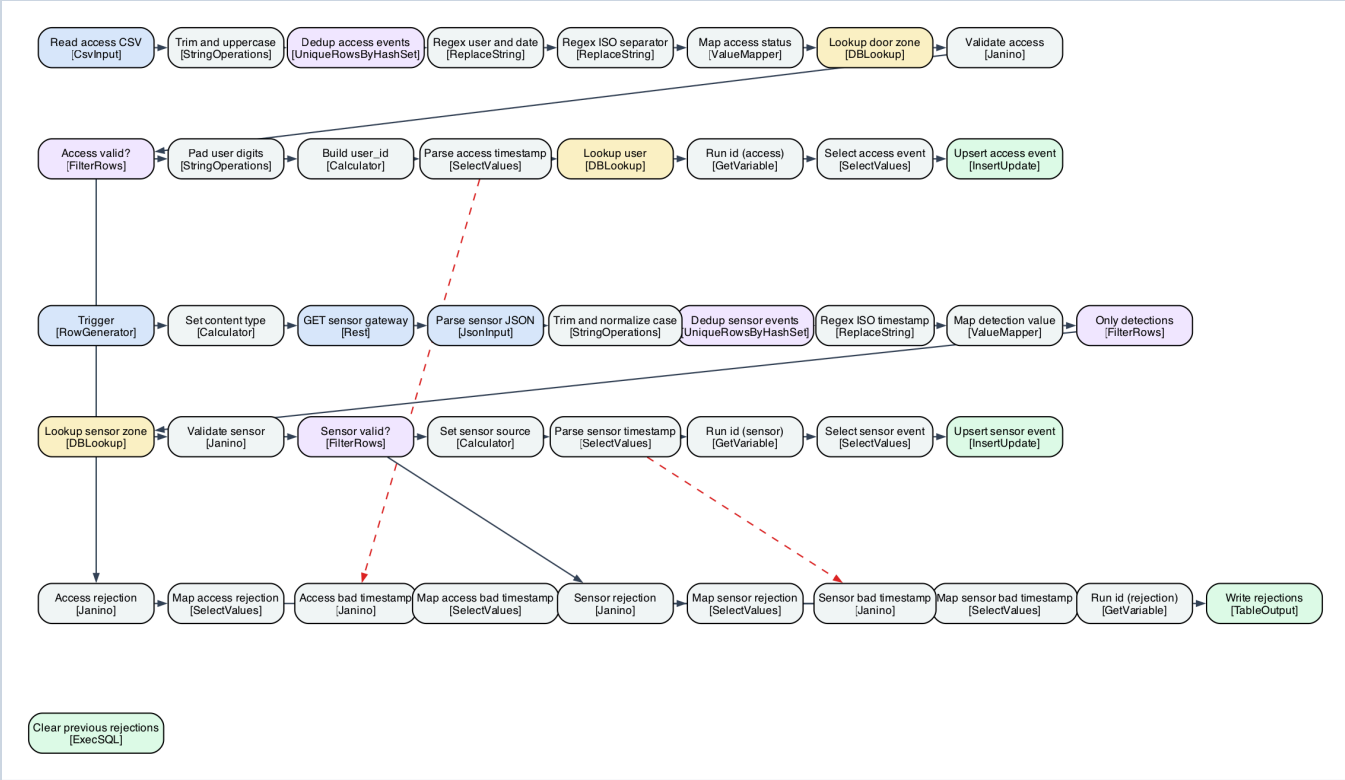


Figura 3 — tr_extract_events (44 steps, 2 hops de erro).

Objetivo. Integrar duas fontes heterogéneas numa tabela única `event`, com limpeza, validação e rastreabilidade das rejeições.

Ramo de acessos (CSV), por ordem:

1. *CSV file input* → *String operations* (*trim* e maiúsculas) → *Unique rows (HashSet)*, que remove 6 duplicados.
2. **Regex** nas datas: `^(\d{2})/(\d{2})/(\d{4})` → `$3-$2-$1` e separador ISO `T` → espaço. Isto unifica três formatos de data.
3. *Value mapper*: estado → tipo de evento. *Database lookup*: porta → zona.
4. Validação em *User Defined Java Expression* + *Filter rows*.
5. Normalização do utilizador (Regex + *padding*). O valor original fica guardado em `raw_user_ref`.
6. Conversão para `Date` com tratamento de erros: uma data impossível segue pelo hop de erro.
7. *Database lookup* do utilizador → *Insert/Update*.

Ramo de sensores (API REST/JSON), por ordem:

1. *Generate rows* → *REST client* (GET) → *JSON input* (JSONPath `$[*].campo`).
2. **Regex** ISO 8601 com e sem fuso.
3. *Value mapper*: `true`/`1`/`on` → deteção. As leituras sem deteção são descartadas.

4. Lookup sensor → zona (a fonte de referência prevalece sobre a zona mal escrita no evento) → validação → *Insert/Update*.

Rejeições. Dois filtros de validação e dois hops de erro alimentam `dq_rejection`, com o registo original e o motivo:

- `MISSING_TIMESTAMP`, `MISSING_USER`, `UNKNOWN_DEVICE`;
- `INVALID_STATUS`, `INVALID_EVENT_TYPE`, `INVALID_TIMESTAMP`.

Um *Execute SQL script* limpa as rejeições anteriores do mesmo `RUN_ID`, para a reexecução não as duplicar.

	Lidos	Duplicados	Sem deteção	Rejeitados	Gravados
Acessos (CSV)	99	6	—	5	**88**
Sensores (REST/JSON)	79	4	9	2	**64**

Os 152 eventos gravados coincidem com o *ground truth*. Uma verificação independente em Python, evento a evento (data, zona, utilizador e tipo), deu 0 diferenças.

4.3 tr_correlate_rules — enriquecimento, correlação e motor de regras

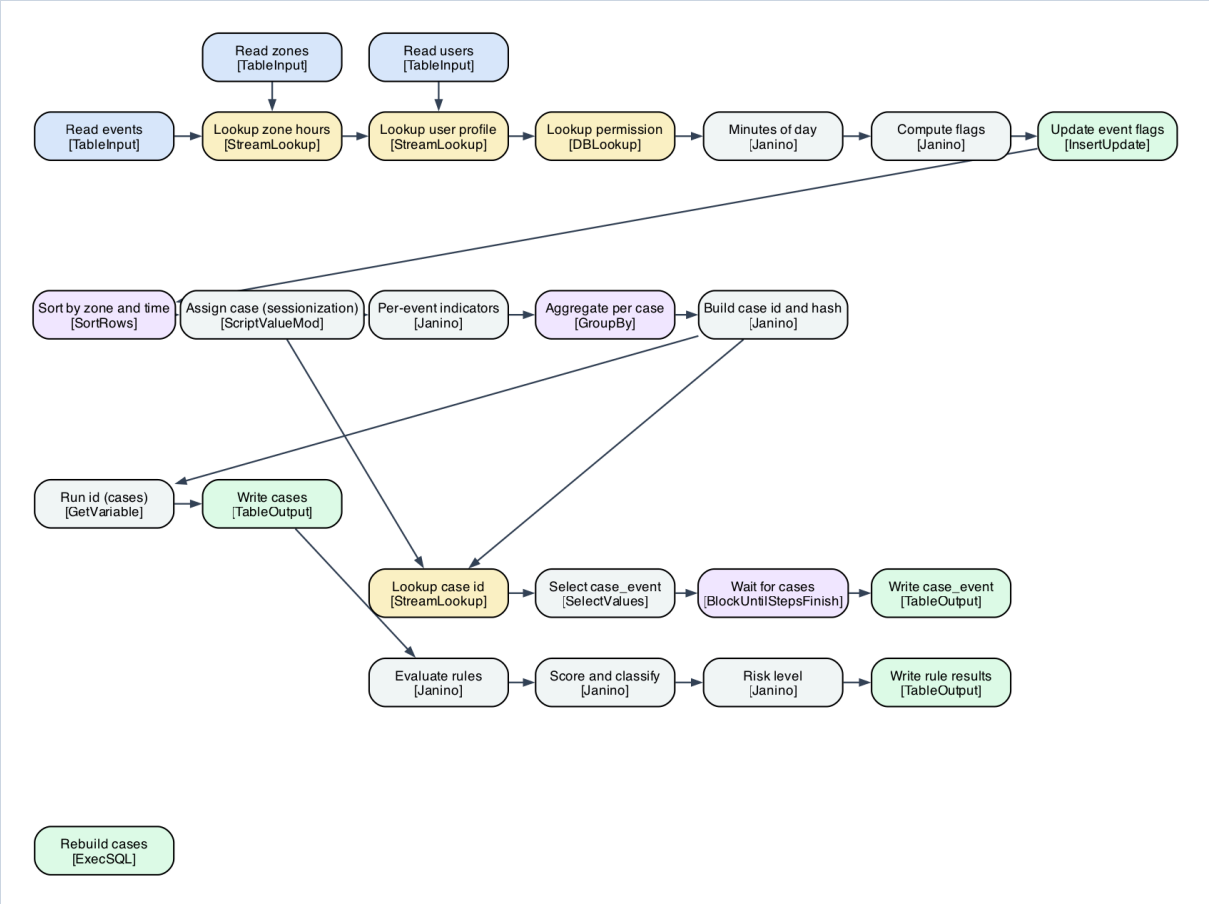


Figura 4 — tr_correlate_rules (25 steps, transacional).

Enriquecimento. Cada evento passa por três lookups:

- *Stream lookup* do horário da zona;
- *Stream lookup* do perfil do utilizador (papel, turno, se está ativo);
- *Database lookup* da permissão utilizador×zona.

Depois, expressões Java calculam as flags do evento:

- `is_off_hours`: zona fechada;
- `is_outside_schedule`: fora do turno do utilizador, incluindo turnos que atravessam a meia-noite;
- `is_unauthorized_zone`, `is_unknown_user` e `is_inactive_user`.

As flags ficam gravadas em `event`.

Correlação (sessionização). Primeiro, *Sort rows* por zona e hora. Depois, um step *Modified JavaScript* com estado entre linhas abre um **novo caso** em três situações:

- a zona muda;
- o intervalo desde o evento anterior é maior do que 5 minutos;
- a duração do caso passaria os 20 minutos.

Os limites são configuráveis (`${CORRELATION_GAP_MIN}`, `${CORRELATION_MAX_SPAN_MIN}`). O limite de duração evita associações em cadeia, que uma janela simples criaria.

O *Group by* agrega cada caso: início e fim, número de eventos, utilizadores distintos, contagens de recusas, presenças, eventos fora de horas e sem permissão, e a lista de IDs. Por fim, uma expressão cria:

- um `case_id` determinístico (`CASE-<zona>-<aaaammdd-hhmmss>`);
- o `events_hash` (MD5 dos IDs), que serve de chave de cache da IA.

A tabela `case_event` liga cada caso aos eventos de origem (rastreabilidade).

Motor de regras (abordagem A).

Regra	Condição	Peso
R1 — Recusa seguida de presença	movimento ou abertura de porta depois de um acesso recusado no mesmo caso	3
R2 — Acesso fora de horas	pelo menos uma tentativa de acesso com a zona fechada	1
R3 — Recusas repetidas	3 ou mais acessos recusados no caso	3
R4 — Atividade sem permissão	pelo menos um acesso **concedido** a uma zona sem permissão	3

A pontuação é a soma dos pesos. O risco é CRITICAL com 6 ou mais, HIGH com 3 a 5, MEDIUM com 2, LOW com 1 e NONE com 0. Um caso é suspeito se a pontuação for pelo menos 1.

Resultado verificado.

- 152 eventos deram **82 casos**, com 152 ligações em `case_event`.
- Cada um dos 15 cenários ficou isolado num caso próprio.
- 11 casos foram considerados suspeitos: 5 HIGH e 6 LOW.
- A reconstrução é feita numa única transação, e os mesmos dados dão os mesmos `case_id`.

4.4 tr_ai_analysis — análise contextual por IA (abordagem B)

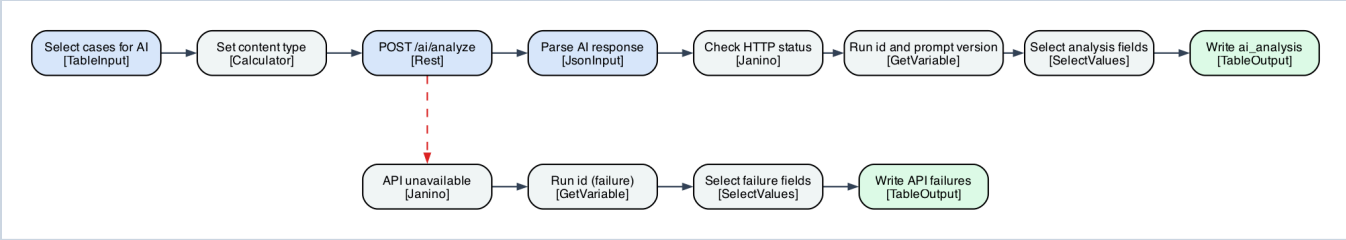


Figura 5 — tr_ai_analysis (12 steps, 1 hop de erro).

Explicação. Um *Table input* com SQL (`etl/sql/ai_case_payloads.sql`) faz três coisas:

1. Seleciona os casos por factos: 15 com sinais, mais 5 de controlo.
2. Exclui os que já foram analisados para a mesma composição (`events_hash`) e versão do *prompt* (cache).
3. Constrói o payload JSON de cada caso (`json_agg`). O payload leva zona, horário, eventos, utilizador, papel, turno, estado e permissão, mas nunca o veredicto das regras.

Segue-se o *REST client* (POST `${API_URL}/ai/analyze`) → *JSON input* → validação do estado HTTP → *Table output* para `ai_analysis`. Se a API estiver em baixo, um hop de erro grava `API_UNAVAILABLE` para cada caso, e nenhum caso se perde.

Serviço `/ai/analyze` (FastAPI → Ollama).

- **Saída estruturada:** o pedido ao Ollama inclui um *JSON schema*, e a lista de evidências está restrita aos IDs do próprio caso.
- **Validação da resposta:** o serviço confirma que as evidências pertencem ao caso e que a resposta é coerente (por exemplo, rejeita "não suspeito" com risco HIGH).
- **Configuração do modelo:** `temperature=0`, `seed=42`, `num_ctx=4096` (limitado pelos 8 GB de RAM) e timeout de 150 s. O *digest* do modelo fica registado em cada análise.
- **Resposta:** é sempre HTTP 200, com um estado (`OK`, `TIMEOUT`, `INVALID_OUTPUT`, `INVALID_INPUT` ou `MODEL_ERROR`). Assim o Pentaho trata as falhas como dados, sem interromper o fluxo.

Resultado verificado.

- 20 casos analisados, 20 com estado OK.
- Duração total de 10 min 52 s, com latência média de 32,1 s por caso no M2.
- As execuções seguintes fazem 0 pedidos (cache).
- Testado com a API desligada (resultado `API_UNAVAILABLE`) e com uma rota inexistente (resultado `MODEL_ERROR`).

4.5 tr_evaluate — regras vs IA vs *ground truth*

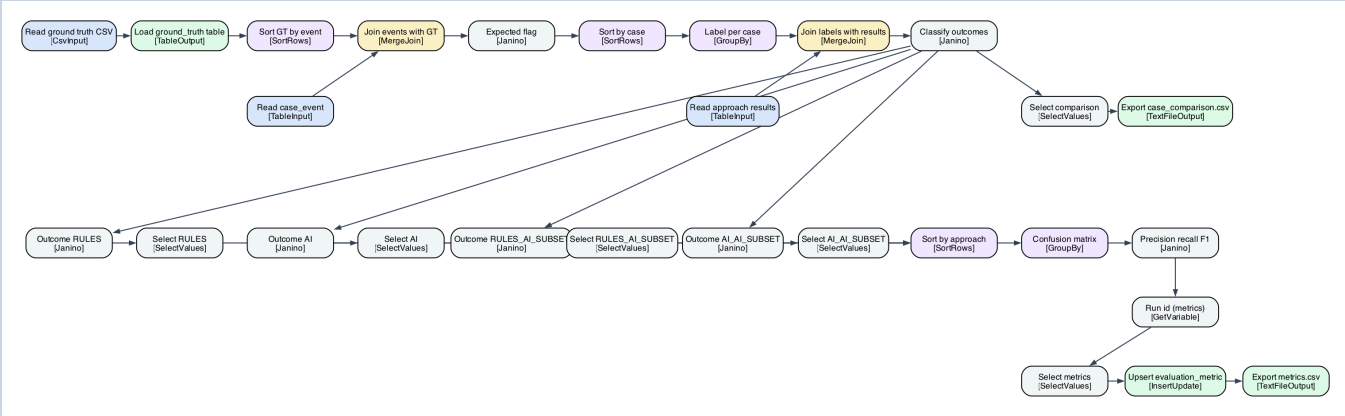


Figura 6 — tr_evaluate (28 steps).

Explicação.

1. O `ground_truth.csv` é carregado (*CSV file input* → *Table output*).
2. Um *Merge join* cruza os eventos de cada caso com as etiquetas. A ordenação é idêntica nos dois lados, com `COLLATE "C"` no SQL, para coincidir com a do Kettle.
3. O *Group by* obtém a etiqueta de cada caso.
4. Um segundo *Merge join* junta as etiquetas aos resultados das regras e da IA. Quatro expressões classificam cada caso em TP, FP, TN ou FN para quatro abordagens:
   - regras e IA, sobre os 82 casos;
   - regras e IA, só sobre o subconjunto enviado à IA.
5. Um *Group by* constrói a matriz de confusão e uma expressão calcula precisão, recall e F1.

Os resultados são gravados em `evaluation_metric` (*Insert/Update*) e exportados para `metrics.csv` e `case_comparison.csv`.

4.6 tr_export_incidents — exportação JSON e CSV

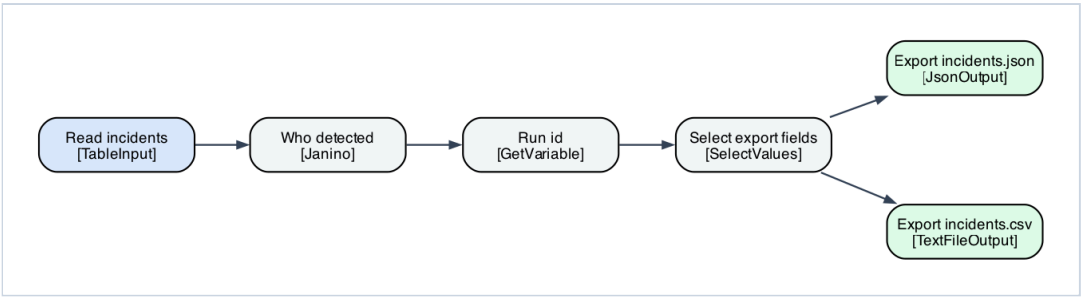


Figura 7 — tr_export_incidents (6 steps).

Exporta os incidentes, isto é, os casos suspeitos segundo as regras e/ou a IA, para `incidents.json` (*JSON output*) e `incidents.csv` (*Text file output*). Cada incidente leva:

- a zona, o intervalo e os eventos de origem;
- o resultado das regras e da IA, e quem o detetou (`BOTH`, `RULES_ONLY` ou `AI_ONLY`);
- a explicação, a ação recomendada e as evidências citadas;
- o modelo e o *digest*.

Na última execução foram exportados 13 incidentes.

4.7 tr_build_notification — composição do e-mail

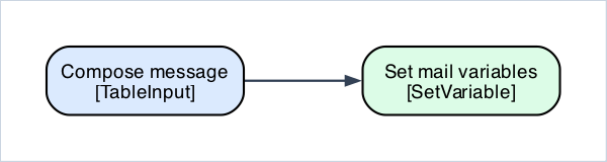


Figura 8 — tr_build_notification (2 steps).

Segue o padrão do exemplo das aulas (`EmailPrevisão.kjb`, aula 06). Um *Table input* lê a base de dados e compõe uma única linha com o tipo de mensagem (resumo ou alerta), o assunto e o corpo em HTML. Um *Set variables* passa esses valores ao Job (`MAIL_SUBJECT`, `MAIL_BODY`), que os usa na entrada *Mail*.

- **Resumo:** contagens, tabela regras vs IA e os 15 incidentes prioritários.
- **Alerta:** a etapa que falhou, a mensagem de erro e o caminho do log.

5. Jobs

5.1 jb_sentinel_main — orquestração completa

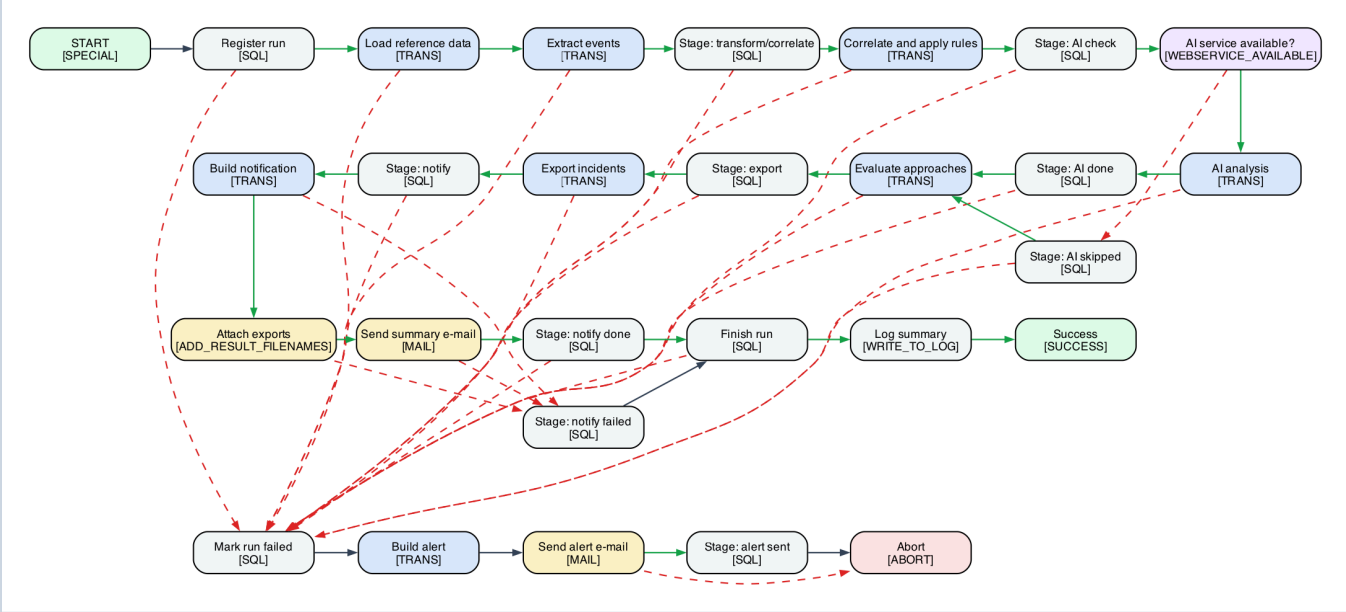


Figura 9 — jb_sentinel_main (28 entradas, 46 hops). Verde: em caso de sucesso; vermelho tracejado: em caso de falha; preto: incondicional.

Explicação. O Job recebe dois parâmetros, `RUN_ID` e `TRIGGER` (cli ou dashboard), e executa as transformações por ordem. Entre elas, entradas *SQL* registam o início e o fim de cada etapa em `etl_run_stage`, com o número de linhas processadas:

1. **Extração e transformação:** `Register run` → `tr_load_reference` → `tr_extract_events` → `tr_correlate_rules`.
2. **IA:** a entrada *Check webservice availability* consulta `${API_URL}/ai/health`.
   - Se o serviço responder, corre `tr_ai_analysis`.
   - Se não, a etapa AI fica SKIPPED e o processo continua sem perder casos. Os casos ficam pendentes para a execução seguinte.
3. **Avaliação e exportação:** `tr_evaluate` → `tr_export_incidents`.
4. **Notificação:** `tr_build_notification` → *Add filenames to result* (anexa `incidents.csv` e `metrics.csv`) → *Mail*. Se o envio falhar, a etapa NOTIFY fica FAILED, mas a execução termina com sucesso, porque os dados já estão corretos.
5. **Fecho:** `Finish run` → *Write to log* → *Success*.
6. **Falha:** qualquer falha numa transformação segue para `Mark run failed` (etapa e execução FAILED), depois para o alerta por e-mail e por fim para *Abort*. O Kitchen termina com código ≠ 0.

5.2 Registo de execuções e logs

- **`etl_run` / `etl_run_stage`:** o estado funcional de cada execução e etapa (PENDING, RUNNING, SUCCEEDED, FAILED ou SKIPPED), com a duração, o número de linhas e as mensagens. O dashboard lê daqui.
- **`log_job`:** a tabela de log nativa do Kettle, configurada no Job.
- **`results/logs/<RUN_ID>.log`:** o log completo do Kitchen, consultável pela API (`/ops/runs/{id}/log`).

Cenário de execução testado	Resultado observado
Execução normal	exit 0; 7 etapas SUCCEEDED (EXTRACT 152, CORRELATE 82, RULES 11, AI, EVALUATE 4, EXPORT, NOTIFY)
Serviço de IA desligado	etapa AI = SKIPPED; o processo continua
Falha forçada numa transformação	etapa e execução FAILED; alerta enviado por e-mail; *Abort*; exit 1
Servidor SMTP indisponível ou credenciais inválidas	NOTIFY = FAILED com o motivo; execução SUCCEEDED (exit 0); erro `AuthenticationFailedException` no log
Reexecução sem dados novos	sem duplicados (O=0, U=0); mesmos 82 casos; IA servida pela cache

5.3 Notificação por e-mail (SMTP)

A entrada *Mail* está configurada como no exemplo da aula 06:

- servidor `smtp.office365.com`, porta 587;
- autenticação segura **TLS**;
- corpo HTML (UTF-8) vindo de uma variável, com "só a mensagem".

Servidor, conta, password, remetente e destinatários são variáveis. Vêm do `.env` ou do formulário do dashboard (secção 5.4). Numa execução lançada pelo dashboard (`run-20261021-123044-e590`), a etapa NOTIFY terminou com "Resumo enviado por e-mail … (SMTP smtp.office365.com:587)", ou seja, o servidor da Microsoft aceitou a mensagem, e o destinatário confirmou que a recebeu. Quando a conta não está configurada ou a autenticação falha, a etapa NOTIFY aparece a vermelho no dashboard com o motivo, e as restantes etapas não são afetadas.

5.4 Orquestração a partir do dashboard (API /ops + Node-RED)

O botão **Executar ETL** do dashboard envia `POST /ops/runs`. A API regista a execução como PENDING e lança o Kitchen com o Job da lista branca. O dashboard mostra depois as etapas (lidas de `etl_run_stage` a cada 3 s) e as últimas linhas do log. O formulário de simulação envia um evento de sensor para a fila do *gateway*; esse evento é processado pelo Pentaho na execução seguinte, sem contornar o ETL.

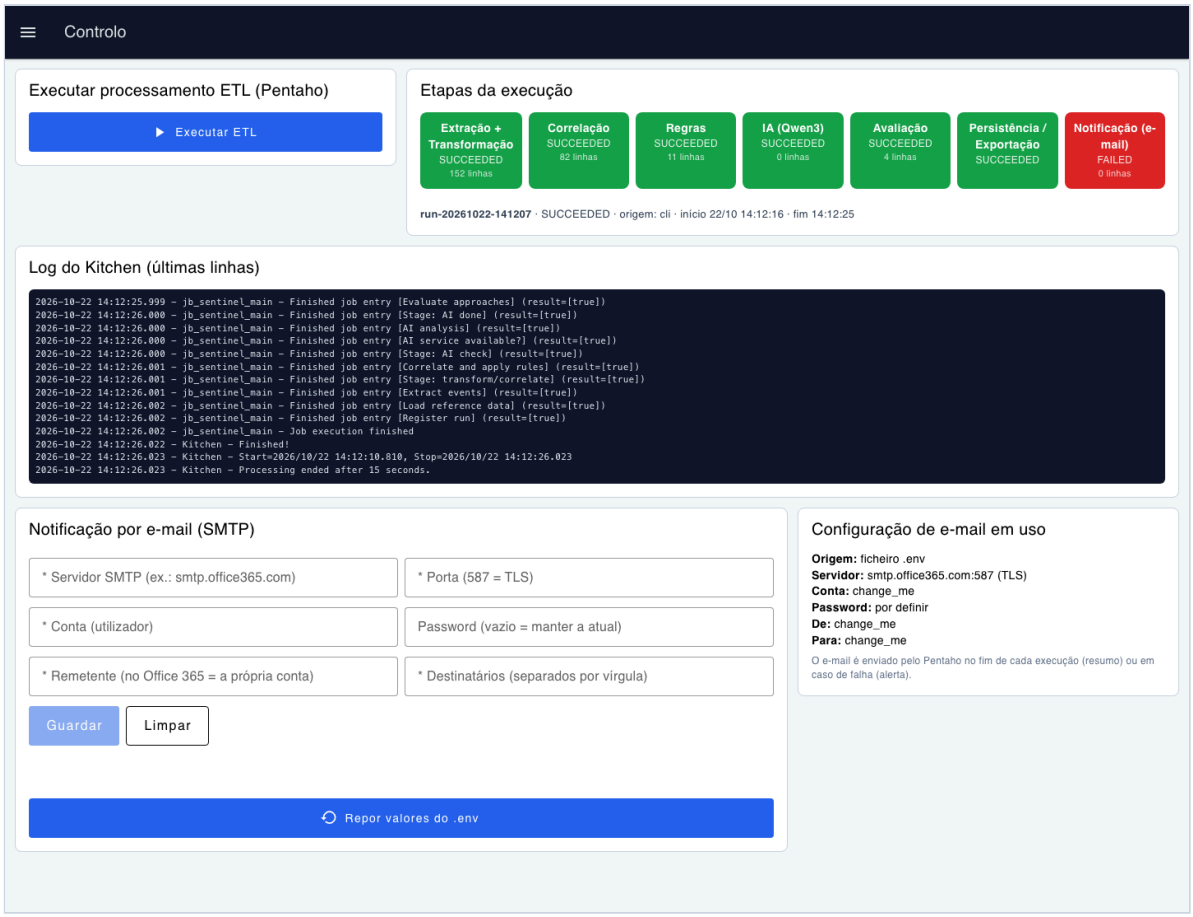


Figura 10 — Página Controlo: execução, etapas, log do Kitchen e configuração do e-mail.

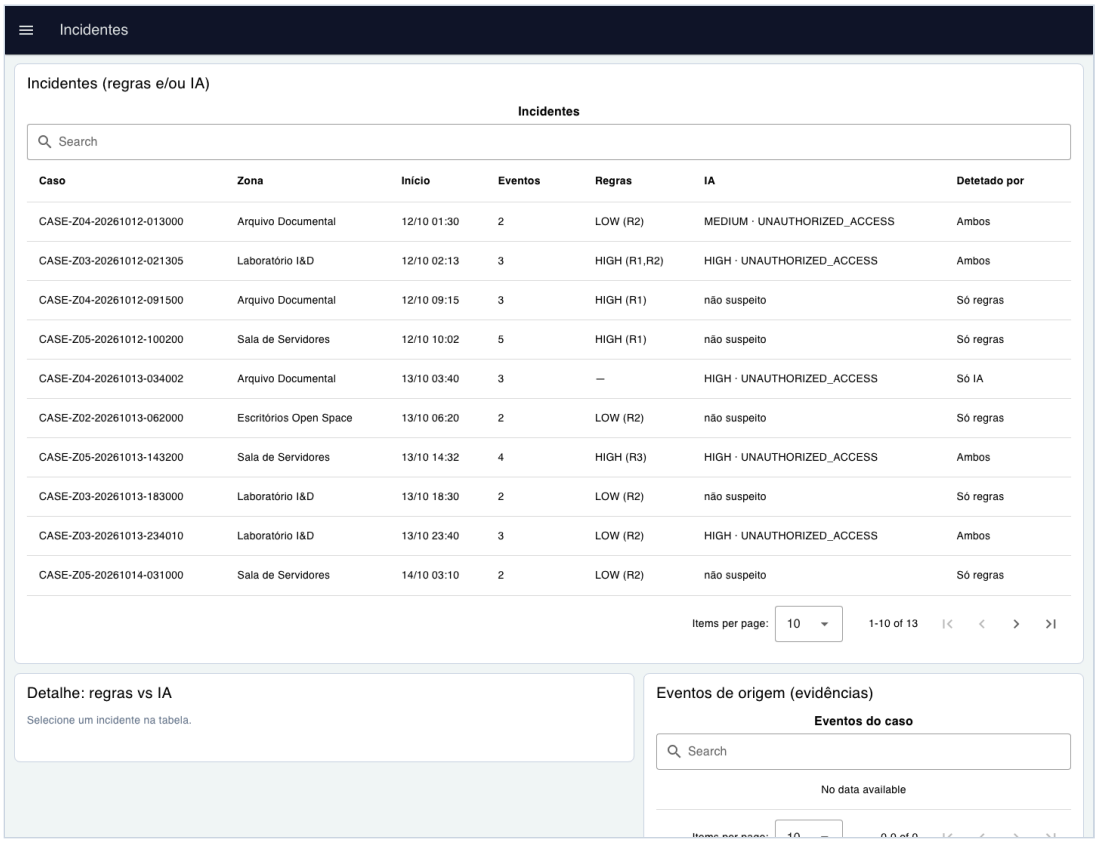


Figura 11 — Incidentes: tabela (regras vs IA), detalhe e eventos de origem (evidências).

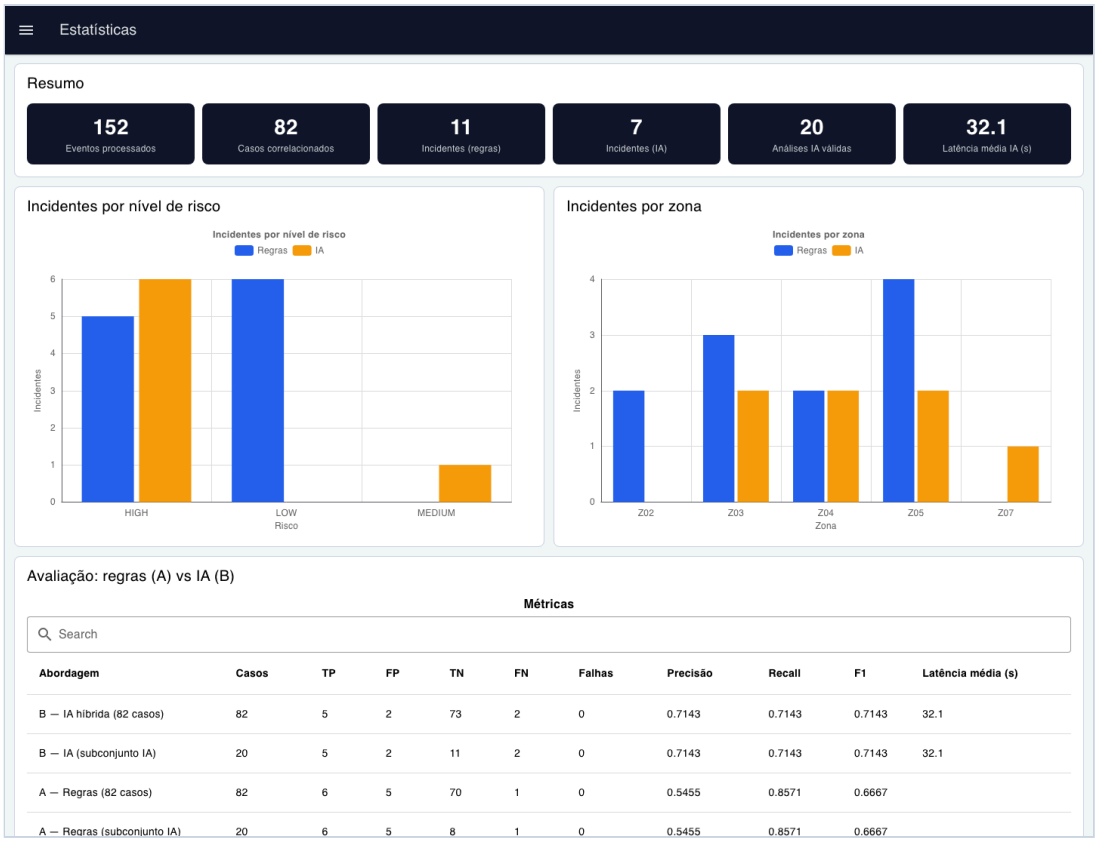


Figura 12 — Estatísticas e métricas de avaliação (regras vs IA).

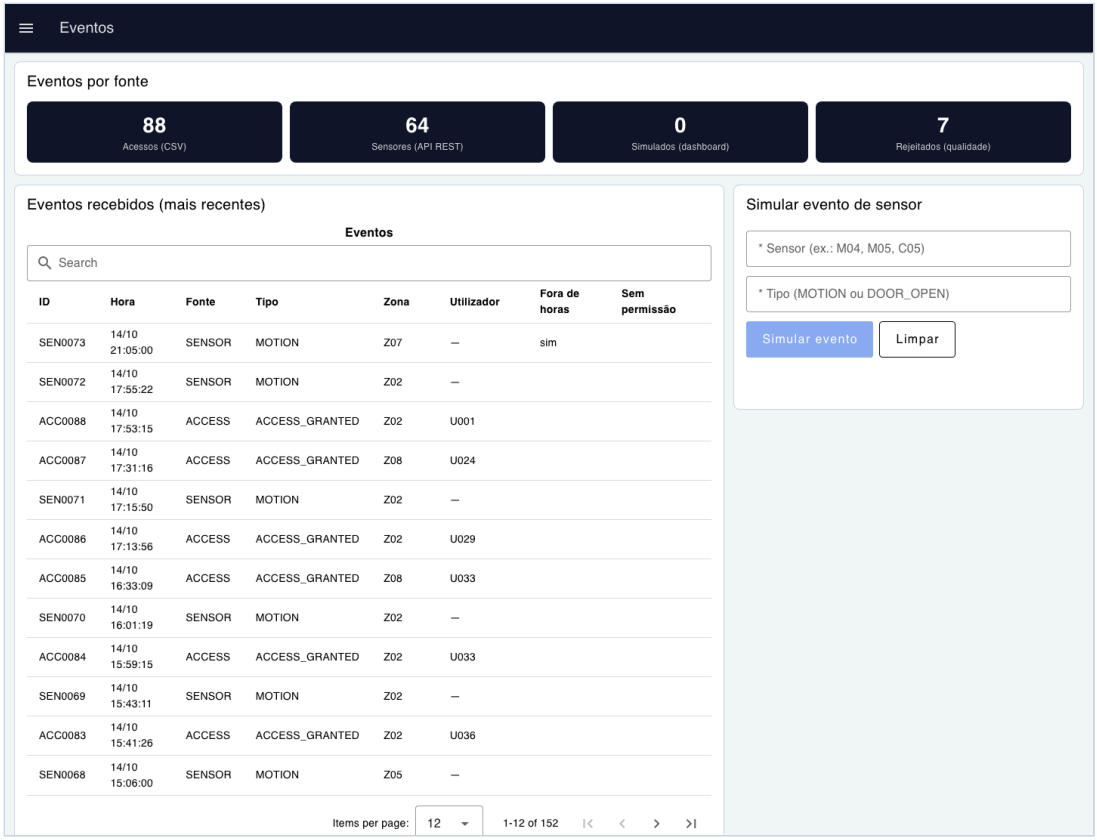


Figura 13 — Eventos por fonte, eventos recebidos e simulação de um evento de sensor.

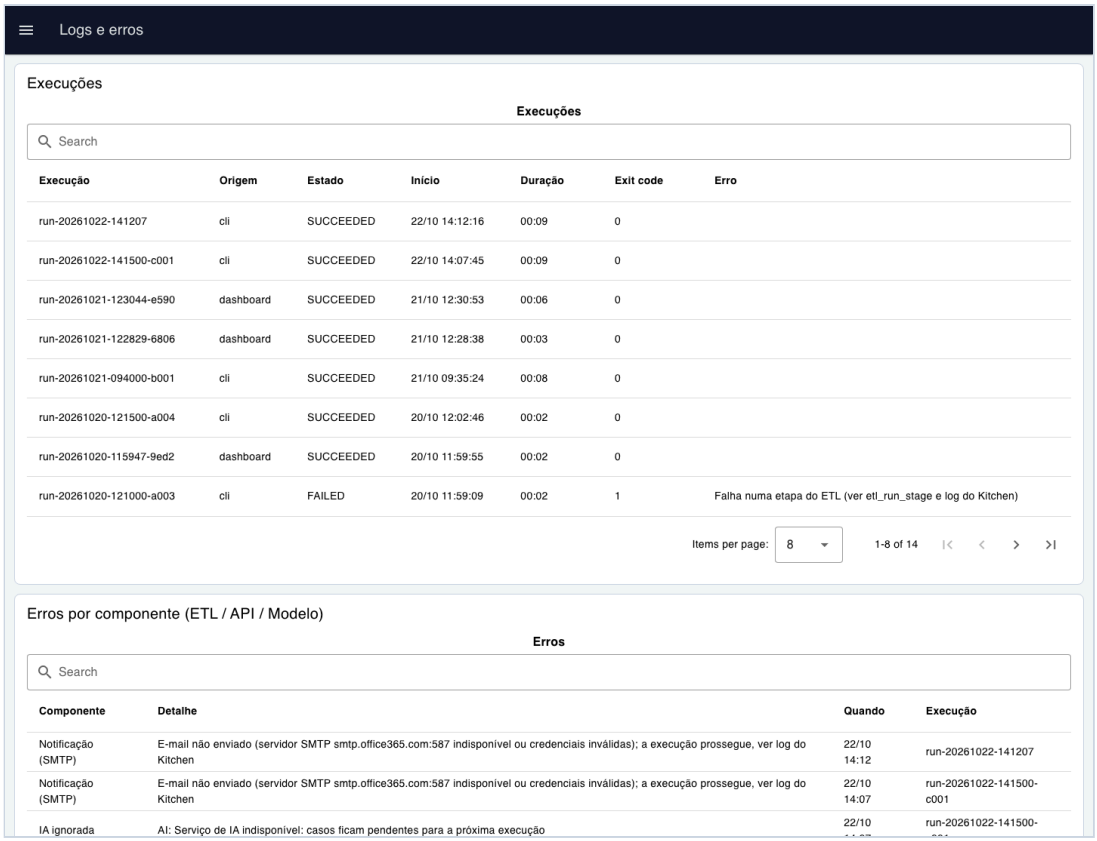


Figura 14 — Execuções, erros por componente (ETL / API / modelo / SMTP) e rejeições de qualidade.

6. Vídeo com demonstração (QR Code)

O vídeo demonstra:

- as transformações e o Job no Spoon;
- a execução a partir do dashboard, com as etapas a atualizar;
- os incidentes com a explicação da IA e as evidências;
- as estatísticas regras vs IA;
- a simulação de um evento;
- o e-mail recebido.



7. Conclusão e trabalhos futuros

7.1 Resultados

Abordagem	Casos	TP	FP	TN	FN	Precisão	Recall	F1
A — Regras	82	6	5	70	1	0,5455	0,8571	0,6667
B — IA (triagem + Qwen3)	82	5	2	73	2	0,7143	0,7143	0,7143

Fonte: `results/evaluation/metrics.csv`. Os falhanços da IA contam-se à parte (0 nesta execução).

Nenhuma das abordagens é melhor do que a outra em tudo:

- **A IA tem maior precisão.** Teve 2 falsos alarmes contra 5 das regras: usou o contexto que as regras ignoram (turno do utilizador, sequência recusa→concessão do mesmo cartão). Também foi a única a detetar o movimento noturno sem acesso (S04).
- **As regras têm maior recall.** Detetaram o ex-colaborador (S05) e o *tailgating* (S06), que o modelo de 4B não relacionou.
- **Custo:** as regras são instantâneas; a IA custa cerca de 32 s por caso. A triagem por factos e a cache tornam esse custo viável.

As abordagens são complementares.

O trabalho cumpre as três fases ETL com o Pentaho como motor central:

- **Fontes:** 4 formatos (CSV, JSON via REST, XML, YAML) e uma base de dados relacional.
- **Transformação:** limpeza com expressões regulares, lookups, joins, agregação e uma correlação com estado.
- **Persistência e exportação:** idempotentes, com rastreabilidade completa até aos eventos de origem.
- **Orquestração:** um Job com registo de etapas, logs, tratamento de falhas e notificação por e-mail.
- **Controlo e visualização:** num dashboard low-code que nunca processa dados.

7.2 Limitações

- **Amostra pequena** (15 cenários e 5 casos de controlo): as métricas são indicativas e não têm significância estatística.
- **Dados sintéticos**, gerados pelo próprio autor. Não há um conjunto de teste separado.
- **IA não totalmente determinística:** a temperatura 0 e a semente fixa reduzem a variação, mas não garantem o determinismo. As explicações têm por vezes vocabulário do português do Brasil.
- **Correlação limitada:** é feita por zona e proximidade temporal. Não relaciona eventos entre zonas adjacentes.

7.3 Trabalhos futuros

- **Combinar as duas abordagens:** alerta automático quando concordam, revisão humana quando discordam.
- **Disponibilização remota das exportações** por FTP/SFTP, a partir do Job.
- **Anonimização ou pseudonimização** de utilizadores e e-mails nas exportações.
- **Ingestão contínua** (MQTT) em vez de execuções em lote.
- **Mais cenários** e um conjunto de teste independente para afinar o *prompt* (nova `PROMPT_VERSION`).

8. Referências

1. L. Ferreira, Ó. Ribeiro — *Integração de Sistemas de Informação*, enunciado do Trabalho Prático I e materiais das aulas (ETL com Kettle; acesso a BD; exemplo de Job com envio de e-mail, aula 06), EST-IPCA, 2026/27.
2. Pentaho Data Integration — documentação: `https://docs.hitachivantara.com/r/en-us/pentaho-data-integration-and-analytics`
3. PostgreSQL 14 — documentação: `https://www.postgresql.org/docs/14/`
4. FastAPI — documentação: `https://fastapi.tiangolo.com/`
5. Pydantic — documentação: `https://docs.pydantic.dev/`
6. Ollama — documentação da API e saídas estruturadas: `https://github.com/ollama/ollama/blob/main/docs/api.md`
7. Qwen3 — modelo `qwen3:4b-instruct`: `https://ollama.com/library/qwen3`
8. Node-RED — documentação: `https://nodered.org/docs/`
9. FlowFuse Dashboard 2.0 — documentação: `https://dashboard.flowfuse.com/`
10. node-red-contrib-postgresql: `https://flows.nodered.org/node/node-red-contrib-postgresql`
11. Graphviz (diagramas): `https://graphviz.org/`